Data Structure

Haiyang Liu

School of Computer & Information Technology,
Beijing Jiaotong University

Chapter 9: Search

Syllabus: Teaching Content

- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- Dynamic Search Tables
  - Binary Search Tree

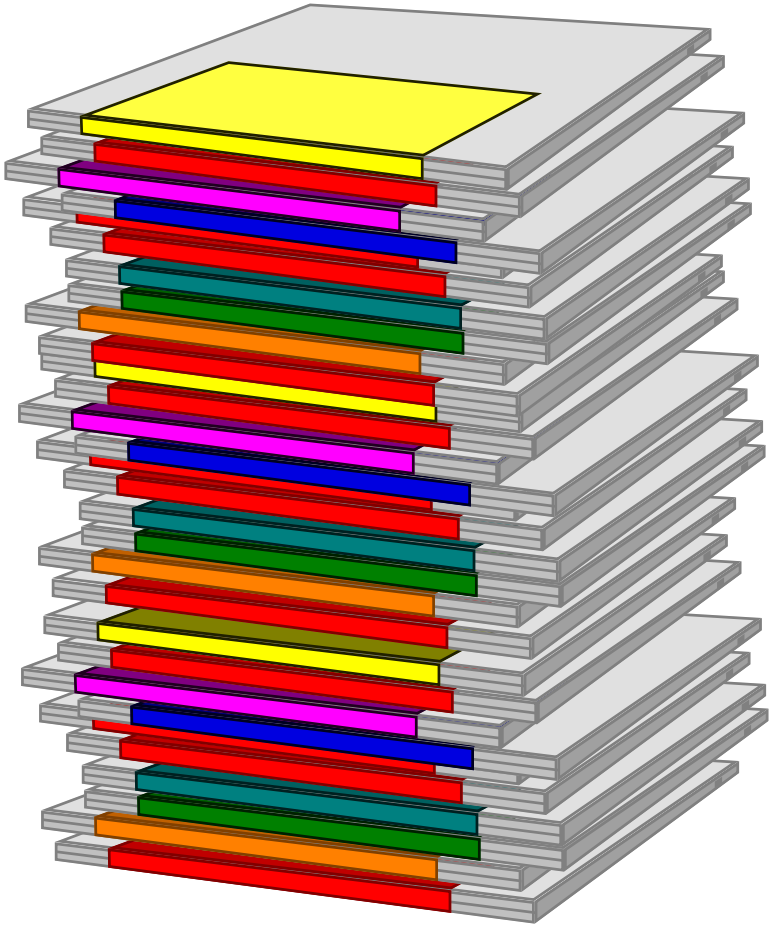
Main Contents of This Chapter

- Basic Concepts of Search Algorithms
- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- Dynamic Search Tables
  - Binary Search Tree

Syllabus: Teaching Content

- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- Dynamic Search Tables
  - Binary Search Tree

Basic Concepts of Search Algorithms



What is a Search Table?

- A search table is a collection of data elements (or records) of the same type.
- Since the data elements in the "collection" have a loose relationship, a search table is a flexible and convenient structure for applications.

Common Operations on Search Tables

- Query whether a "specific" data element exists in the search table;
- Retrieve various attributes of a "specific" data element;
- Insert a data element into the search table;
- Delete a data element from the search table.

Search Tables Can Be Divided into Two Categories

- Static Search Table:

  A search table that only performs query and retrieval operations is a static search table.

- Dynamic Search Table:

  Sometimes, after a query, it is necessary to insert a data element that is "not in the search table" into the table; or, delete a data element that is "in the search table" based on the query result. Such a table is a dynamic search table.

Several Concepts

- Keyword: The value of a certain data item used to identify a data element (or record) is called a keyword.
- Primary Keyword: If a keyword can uniquely identify a record, then this keyword is called a primary keyword.
- Secondary Keyword: Conversely, a keyword used to identify several records is a secondary keyword.

Search

- According to a given value, determine a data element or record in the search table whose keyword is equal to the given value.
- If such a record exists in the search table, it is called a "successful search". The search result provides the information of the entire record or indicates the position of the record in the search table.
- Otherwise, it is called an "unsuccessful search". The search result provides a "null record" or a "null pointer".

How to Perform Search?

- The search method depends on the structure of the search table, i.e., the relationship by which the data elements in the table are organized.
- Since there is no obvious organizational rule among the data elements in the search table, it is not convenient to perform search.
- To improve search efficiency, it is necessary to artificially add some definite relationship among the elements in the search table. In other words, **represent the search table with another structure**.

Main Contents of This Chapter

- Basic Concepts of Search Algorithms
- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- Dynamic Search Tables
  - Binary Search Tree

Definition of Static Search Table (Abstract Data Type)

- ADT StaticSearchTable {
- (1) Data Object
- (2) Data Relationship
- (3) Basic Operations
- **} ADT StaticSearchTable**

Definition of Static Search Table (Abstract Data Type)

data object D: D is a collection of data elements with identical characteristics. Each element contains a unique identifier key of the same type.

Data Relationship R: Data elements belong to the same set.

basic operation P: Create(&ST, n);

Destroy(&ST);

Search(ST, key);

Traverse(ST, Visit());

Basic Operations

- Create(&ST, n)：Operation Result: Construct a static search table ST containing n data elements.
- Destroy(&ST)：Precondition: The static search table ST exists.Operation Result: Destroy the table ST.
- Search(ST, key)：Precondition: The static search table ST exists, and key is a given value of the same type as the keywords of the elements in the search table.Operation Result: If there is a data element in ST whose keyword is equal to key, the function returns the value of the element or its position in the table; otherwise, it returns "null".
- Traverse(ST, Visit())：Precondition: The static search table ST exists, and Visit is an application function for operating on elements.Operation Result: Call the function Visit() on each element of ST exactly once in a certain order. If Visit() fails at any point, the operation fails.

Sequential Storage Structure

- Assume the sequential storage structure of a static search table is defined as

```
typedef  struct {
    ElemType *elem;  //Base address of the data element storage space,
//allocated according to the actual length when building the table, with unit 0 left empty
    int  length;            // Length of the table
} SSTable;
```

- The definition of the data element type is:

```
typedef struct {
    keyType  key;           // Keyword field
        … …                 // Other attribute fields
} ElemType ;
```

Macro Definitions (Conventions for Comparing Two Keywords)

For Numeric Keywords

- #define EQ(a, b) ((a)= = (b))
- #define LT(a, b) ((a) < (b))
- #define LQ(a, b) ((a) < = (b))

For String-Type Keywords

- #define EQ(a, b) (! strcmp((a) , (b)))
- #define LT(a, b) (strcmp((a) , (b)) < 0)
- #define LQ(a, b) (strcmp((a) , (b)) <= 0)

Static Search Tables

1、 Sequential Search Table

$$ASL_{ss} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2}$$

2、 Sorted Search Table

$$ASL_{bs} = \frac{1}{n}\sum_{i=1}^{n}C_i = \frac{1}{n}\left[\sum_{j=1}^{h} j \times 2^{j-1}\right] = \frac{n+1}{n}\log_2(n+1) - 1$$

3、 Static Search Tree Table

4、 Indexed Sequential Table

Sequential Search Table

- Represent a static search table with a sequential table or a linear linked list.

（1）Review the sequential table search process：

k

ST.elem

	21	37	88	19	92	05	64	56	80	75	13	
0	1	2	3	4	5	6	7	8	9	10	11	

ST.Length

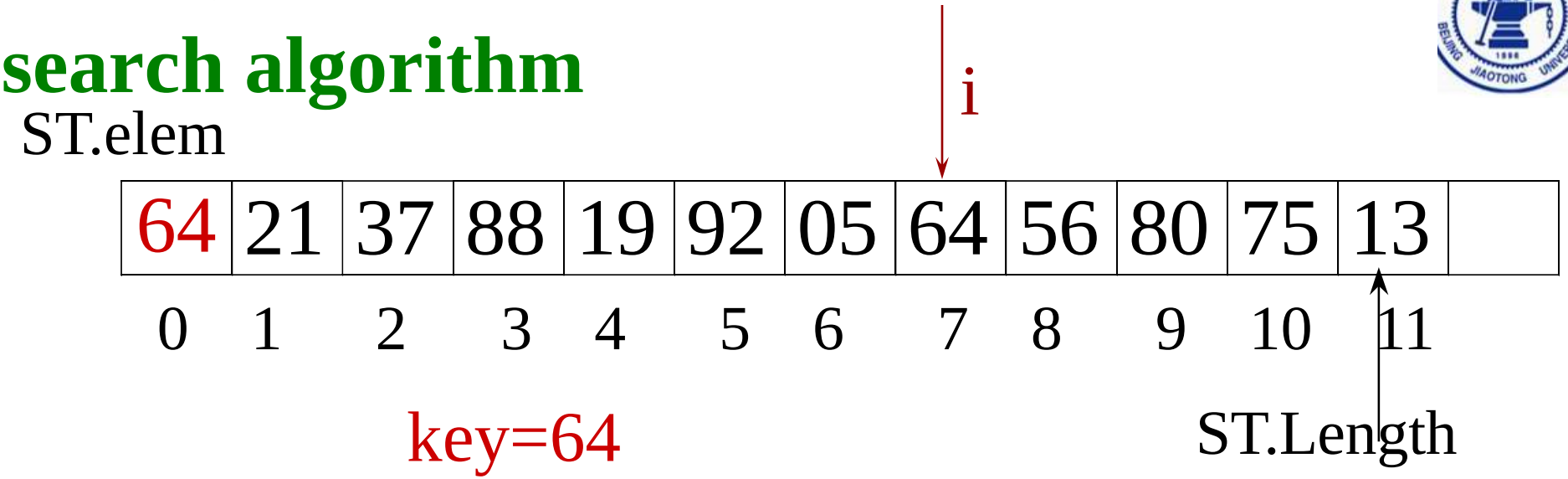
Suppose the given value e=64,

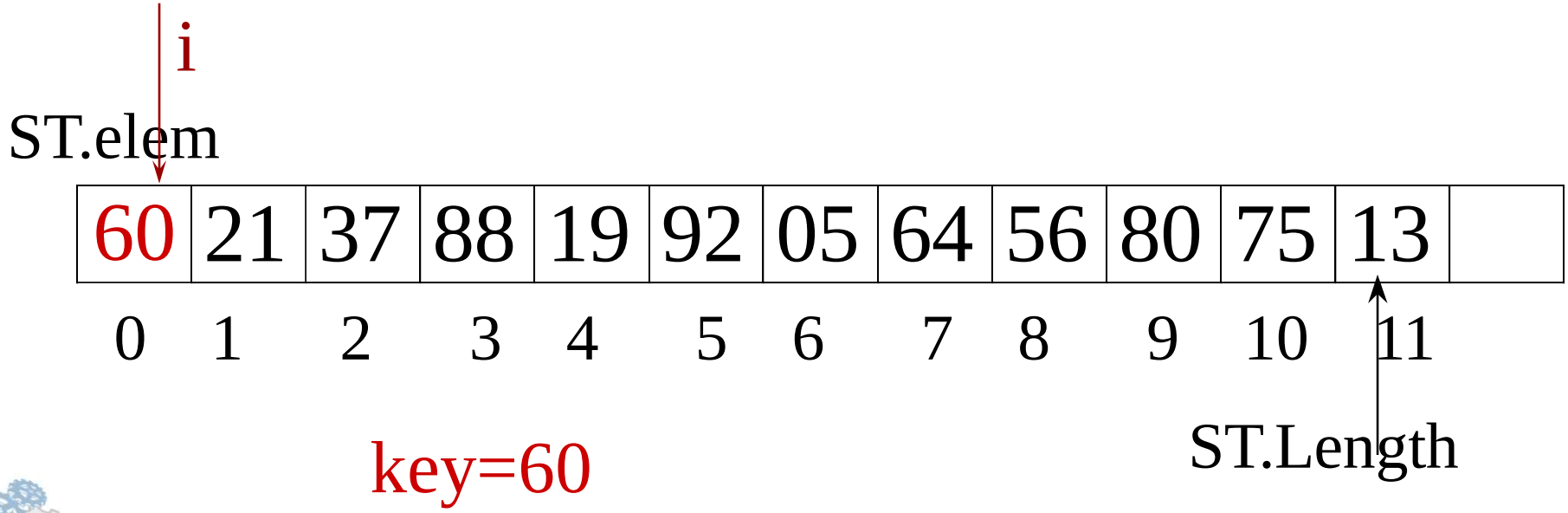
Require ST.elem[k] = e, Find: k = ?

（2）The algorithm for checking from front to back...

```
int location( SqList L, ElemType& e,
      Status (*compare)(ElemType, ElemType)) {
    k = 1;
    p = L.elem;
    while ( k<=L.length &&
                    !(*compare)(*p++,e))
        k++;
    if ( k<= L.length)  return k;
    else  return 0;
} //location
```

…（2）The forward-backward search algorithm





©Haiyang Liu, 2025

（3）The backward search algorithm

```
int Search_Seq(SSTable ST,
                                KeyType key) {
// In the sequential table ST, sequentially search for the data element
 // whose keyword is equal to key. If found, the function returns
 //  the position of the element in the table; otherwise, it returns 0.
   ST.elem[0].key = key;        // "Sentinel"
   for (i=ST.length; ST.elem[i].key!=key;  --i);
                        // Search from back to front
   return i;            // When not found, i is 0
} // Search_Seq
```

In the algorithm that searches from back to front, before the search, the 0th element is set to the search element key. This avoids checking whether the entire table has been searched at each step during the search process. ST.elem[0] plays the role of a sentinel.

The above improvement in programming technique reduces the average search time by half when the length of the search table exceeds 1000.

(4) Analyze the time performance of sequential search

Find the average search length of the algorithm

(Average Search Length)

The expected number of key comparisons required to locate a record in a table is called the average search length of the search algorithm when the search is successful.

The number of comparisons when the search fails is n+1.

For a table containing n data elements, the average search length when the search is successful is:

$$ASL = \sum_{i=1}^{n} P_i C_i$$

Where:Pi is the probability of searching for the i-th data element, and

$$\sum_{i=1}^{n} P_i = 1$$

Ci is the number of comparisons needed when finding the i-th data element.

For a sequential table, C_i depends on the position of the element being searched:

- When searching for record A[n], only one comparison is needed;
- When searching for record A[1], n comparisons are needed;
- When searching for record A[i], n-i+1 comparisons are needed.

$$ASL = nP_1 + (n-1)P_2 + \ldots + 2P_{n-1} + P_n$$

In the case of equal-probability search, $P_i = \frac{1}{n}$

The average search length for sequential table search is:

$$ASL_{ss} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2}$$

Main Contents of This Chapter

- Basic Concepts of Search Algorithms
- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- Dynamic Search Tables
  - Binary Search Tree

2、Sorted Search Table

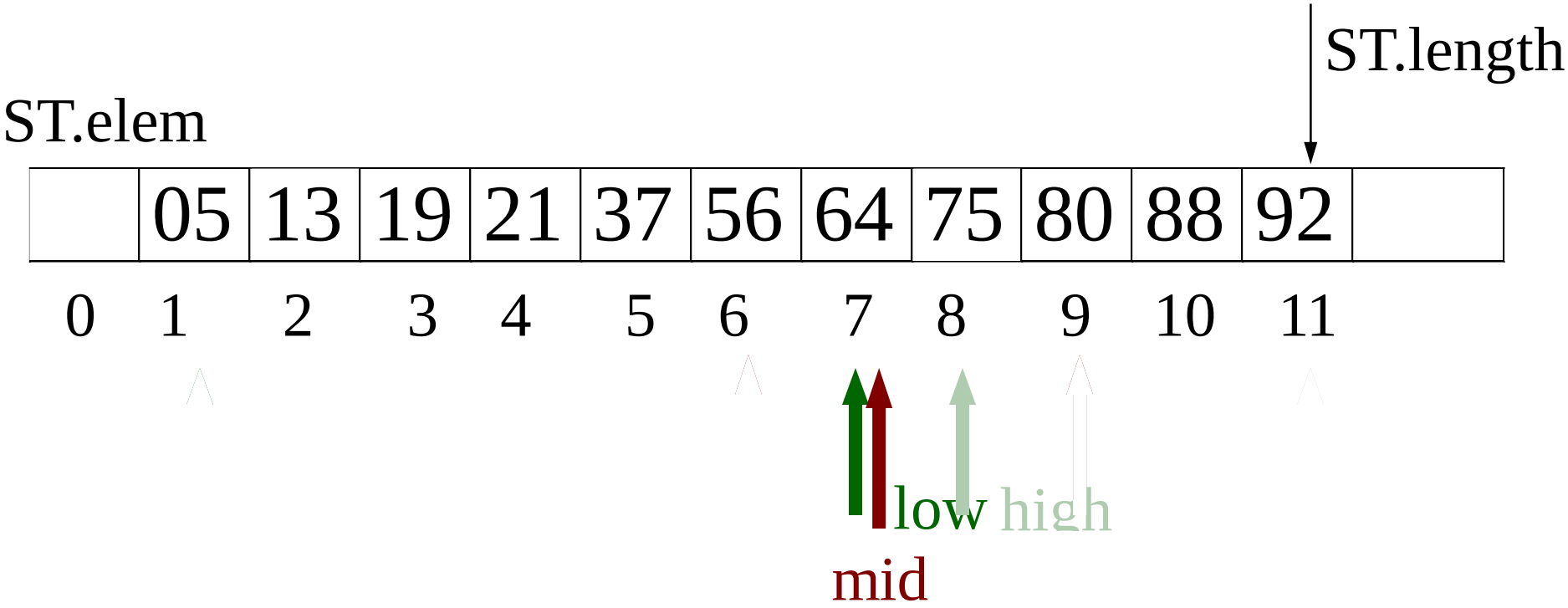
The search algorithm for the aforementioned sequential search table is simple, but its average search length is relatively large, making it particularly unsuitable for search tables with a large number of elements.

If a static search table is represented by a sorted table, the search process can be performed based on "binary search".

(1) The fundamental concept of binary search

- If the data elements in the search table are sorted by key (assuming ascending order), you can skip through the table instead of comparing them sequentially.
  - First compare the keyword value with the "middle position"; if equal, the search is successful.
  - If the given value is greater than the middle position keyword value, continue the binary search in the latter half.
  - Otherwise, perform a binary search in the first half.
- Binary search is only applicable to ordered lists organized in sequential storage structures.

For example, the search process for key=64 is as follows:



low indicates the lower boundary of the search range

high Indicates the upper bound of the search interval.

mid = (low+high)/2 （rounded down）

（2）Binary Search Algorithm

- Let the lower boundary of the area containing the target element be low and the upper boundary be high. Then the midpoint is calculated as mid=(low + hig)DIV2.
  - If the middle element value matches the given value, the search is successful.
  - If the middle element value is greater than the given value, then in the range

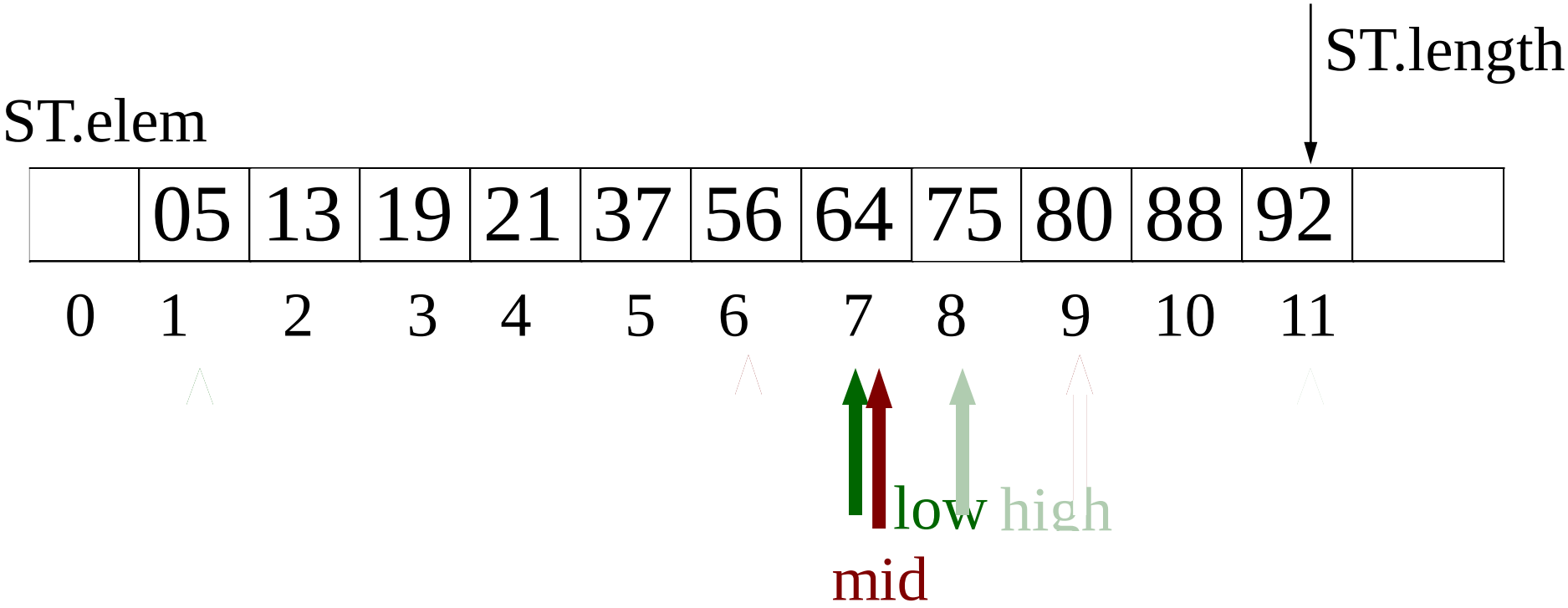
  [low, mid-1] perform binary search
  - If the middle element value is less than the given value, then in the range

  Perform binary search in [mid+1, hig]

(1) The fundamental concept of binary search

- If the data elements in the search table are sorted by key (assuming ascending order), you can skip through the table instead of comparing them sequentially.
  - First compare the keyword value with the "middle position"; if equal, the search is successful.
  - If the given value is greater than the middle position keyword value, continue the binary search in the latter half.
  - Otherwise, perform a binary search in the first half.
- Binary search is only applicable to ordered lists organized in sequential storage structures.

For example, the search process for key=64 is as follows:



low indicates the lower boundary of the search range

high Indicates the upper bound of the search interval.

mid = (low+high)/2 （rounded down）

（2）Binary Search Algorithm

- Let the lower boundary of the area containing the target element be low and the upper boundary be high. Then the midpoint is calculated as mid=(low + hig)DIV2.
  - If the middle element value matches the given value, the search is successful.
  - If the middle element value is greater than the given value, then in the range

  [low, mid-1] perform binary search
  - If the middle element value is less than the given value, then in the range

  Perform binary search in [mid+1, hig]

Binary search algorithm

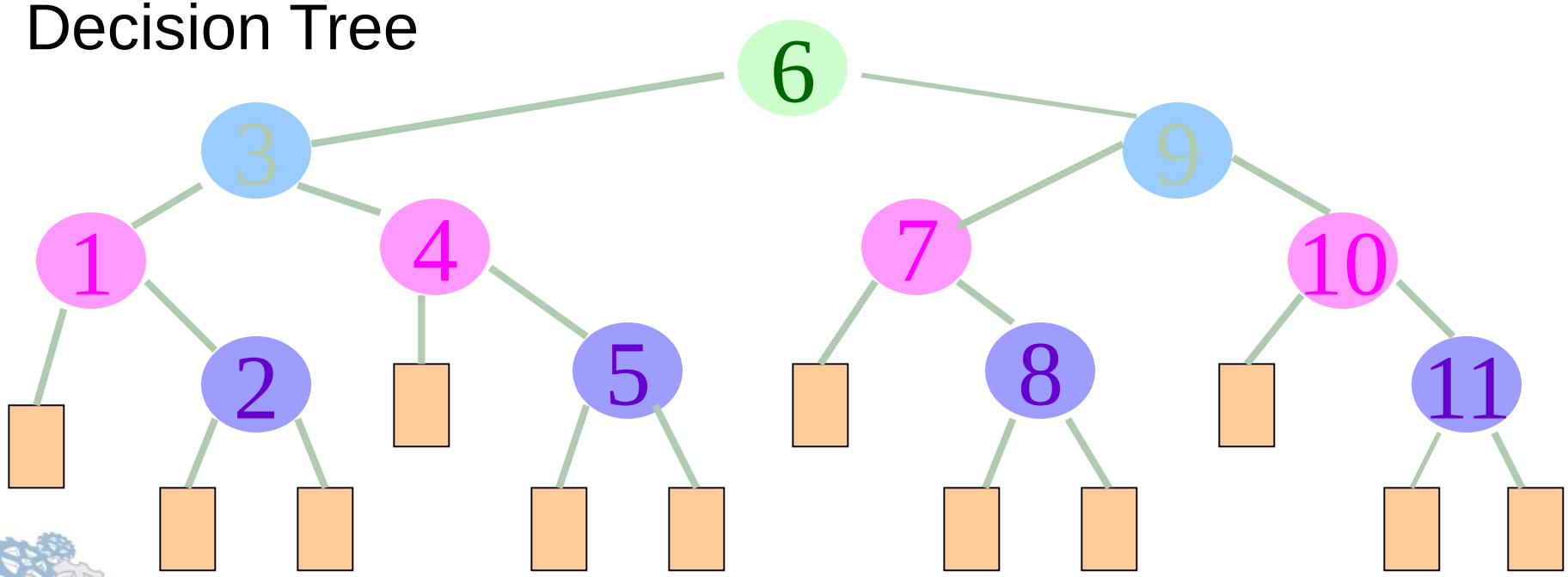
```
int Search_Bin ( SSTable ST, KeyType key ) {
   low = 1;  high = ST.length; // Initial values for the interval
   while (low <= high) {
      mid = (low + high) / 2;
      if （EQ (key , ST.elem[mid].key) )
        return  mid;          // Find the element to check
      else  if ( LT (key , ST.elem[mid].key) )
        high = mid - 1;
// Continue searching in the first half of the interval
      else  low = mid + 1;
//  Continue searching in the latter half of the interval
   }
   return 0;  // The element is not found in the sequential list

} // Search_Bin
```

(3)Analyze the Time Performance (ASL) of Binary Search

First, let's look at a specific case. Assume n=11

i	1	2	3	4	5	6	7	8	9	10	11
Ci	3	4	2	3	4	1	3	4	2	3	4

Decision Tree



The above search process can be described using a binary tree (also called a decision tree). Each circular node (internal node) in the tree represents a record in the table, and the value in the circular node represents the position of the record in the table. Square nodes (external nodes) are artificially added.

The number of comparisons when the search is successful does not exceed the depth of the decision tree. The number of comparisons is equal to the number of internal nodes on the path.

The process when the search is unsuccessful follows the path from the root node to an external node (i.e., the depth of the tree). The number of comparisons is equal to the number of nodes on this path, and it does not exceed $\lfloor \log_2 n \rfloor + 1$.

The depth of a decision tree with n nodes is the same as that of a complete binary tree with n nodes.

Given n=2^h-1 and equal search probabilities (P_i=1/n), the decision tree is a full binary tree of depth h. With 2^h-1 nodes at each level, the average search length (ASL) is:

$$ASL_{bs} = \frac{1}{n}\sum_{i=1}^{n} C_i = \frac{1}{n}\left[\sum_{j=1}^{h} j \times 2^{j-1}\right] = \frac{n+1}{n}\log_2(n+1) - 1$$

When n>50, an approximate result is obtained.

$$ASL_{bs} \approx \log_2(n+1) - 1$$

Main Contents of This Chapter

- Basic Concepts of Search Algorithms
- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- **Dynamic Search Tables**
  - Binary Search Tree

Characteristics of Dynamic Search Table

- The table structure is generated dynamically during the search process. If a record with the specified key exists in the table, the search is successful and the result is returned. Otherwise, a record with the specified key is inserted.

Definition of the dynamic search table for abstract data types

ADT DynamicSearchTable {

data object D:

D is a collection of data elements with identical characteristics.
Each data element contains a unique keyword of the same type.

Data Relationship R: Data elements belong to the same set.

basic operationP：

InitDSTable(&DT)

DestroyDSTable(&DT)

SearchDSTable(DT, key);

InsertDSTable(&DT, e);

DeleteDSTable(&DT, key);

TraverseDSTable(DT, Visit());

}ADT DynamicSearchTable

InitDSTable(&DT)

Operation result: Construct an empty dynamic search table DT.

DestroyDSTable(&DT)

Precondition: The dynamic search table DT exists.

Operation Result: Destroy the dynamic search table DT

SearchDSTable(DT, key)

Precondition: The dynamic search table DT exists, and key is a given value of the same type as the keywords.

Operation Result: If there is a data element in DT whose keyword is equal to key, the function returns the value of the element or its position in the table; otherwise, it returns "null".

InsertDSTable(&DT, e)

initial condition : The dynamic search table DT exists, and e is the data element to be inserted.

Operation result: If no data element with the keyword equal to e.key exists in DT, insert e into DT.

DeleteDSTable(&DT, key)

initial condition : The dynamic search table DT exists, and key is a given value of the same type as the key.

Operation result: If the DT contains a data element with the key equal to key, it will be deleted.

TraverseDSTable(DT, Visit())

initial condition : The dynamic search table DT exists, and Visit is an application function for node operations.

Operation result: Call the Visit() function for each node in DT exactly once and at most once in a specific order. If Visit() fails, the operation will fail.

Main Contents of This Chapter

- Basic Concepts of Search Algorithms
- Static Search Tables (Creation and Search)
  - Sequential Table
  - Sorted Table
- Dynamic Search Tables
  - Binary Search Tree

Binary search tree

1. Definition

2. Search algorithm

3. Insert algorithm

4. Delete algorithm

5. Performance analysis

1.Binary Search Tree (BST) definition:

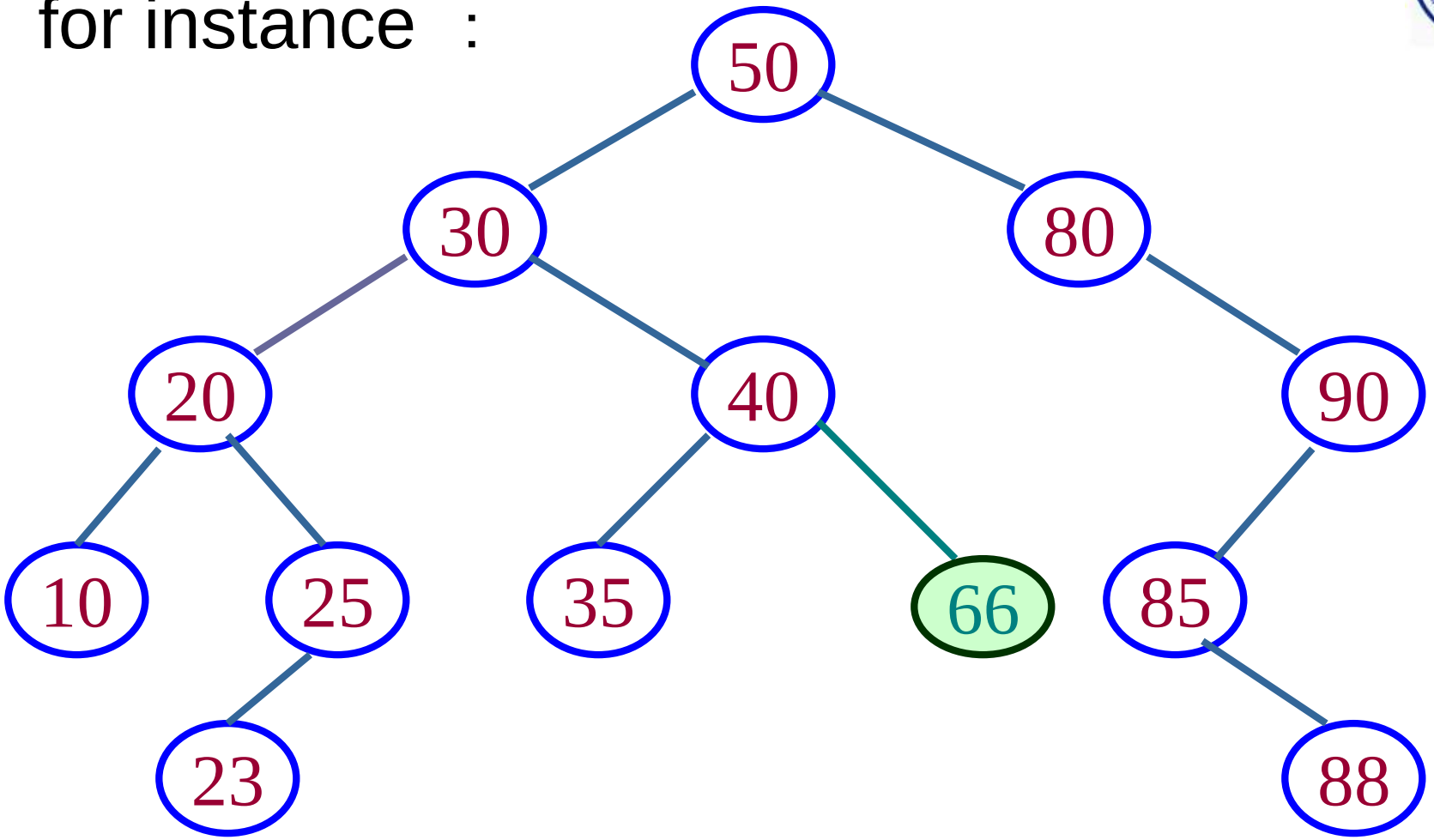
A binary search tree is either an empty tree or a binary tree with the following properties:

(1) If its left subtree is not empty, then all nodes in the left subtree have values less than the root nodes value.

(2) If its right subtree is not empty, then all nodes in the right subtree have values greater than the root nodes value.

(3) Both its left and right subtrees are binary search trees.

for instance :



It is **not** a binary sort tree.

©Haiyang Liu, 2025

A binary linked list is typically used as the storage structure for a binary sort tree.

```
typedef struct BiTNode {// Node structure
    TElemType      data;
    struct BiTNode  *lchild, *rchild;
    // Left and right child pointers
} BiTNode, *BiTree;
```

2. Binary search tree search algorithm:

If the binary search tree is empty, the search will fail.

otherwise，

1) If the given value **equals** the root nodes key, then

Found;

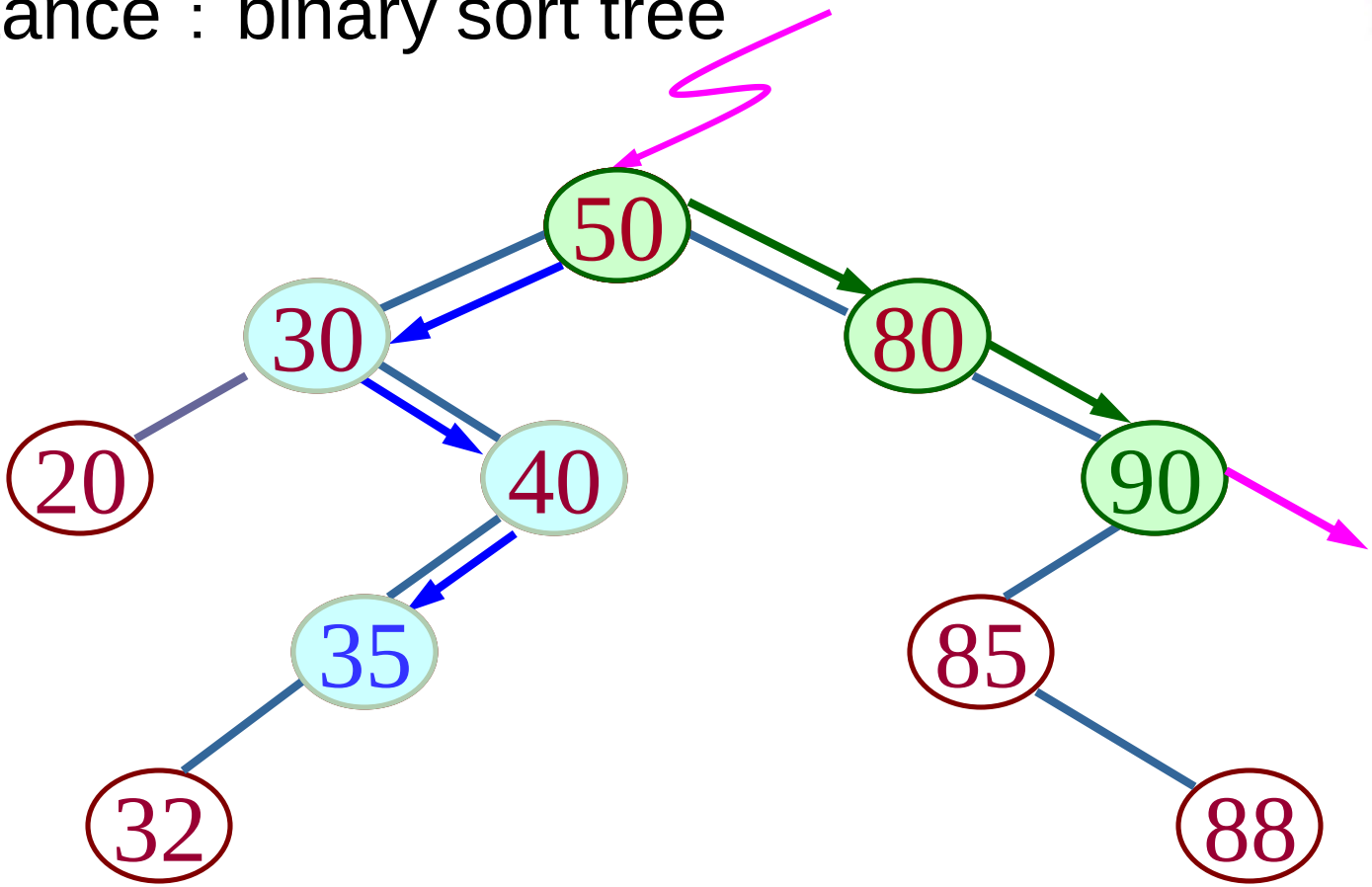
2) If the given value is **less than** the root nodes key, then

Continue searching in the left subtree.

3) If the given value is **more than** the root nodes key, then

Continue searching the right subtree.

for instance：binary sort tree



search key

== 50 , 35 , 90 , 95 ,

©Haiyang Liu, 2025

As can be seen from the above search process,

During the search, a search path was generated:

Starting from the root node, follow the left or right branch down the hierarchy until the key equals the given value;

— Found

or

Starting from the root node, traverse the left or right branch layer by layer until the pointer points to an empty tree.

— Search failed

The algorithm is described as follows:

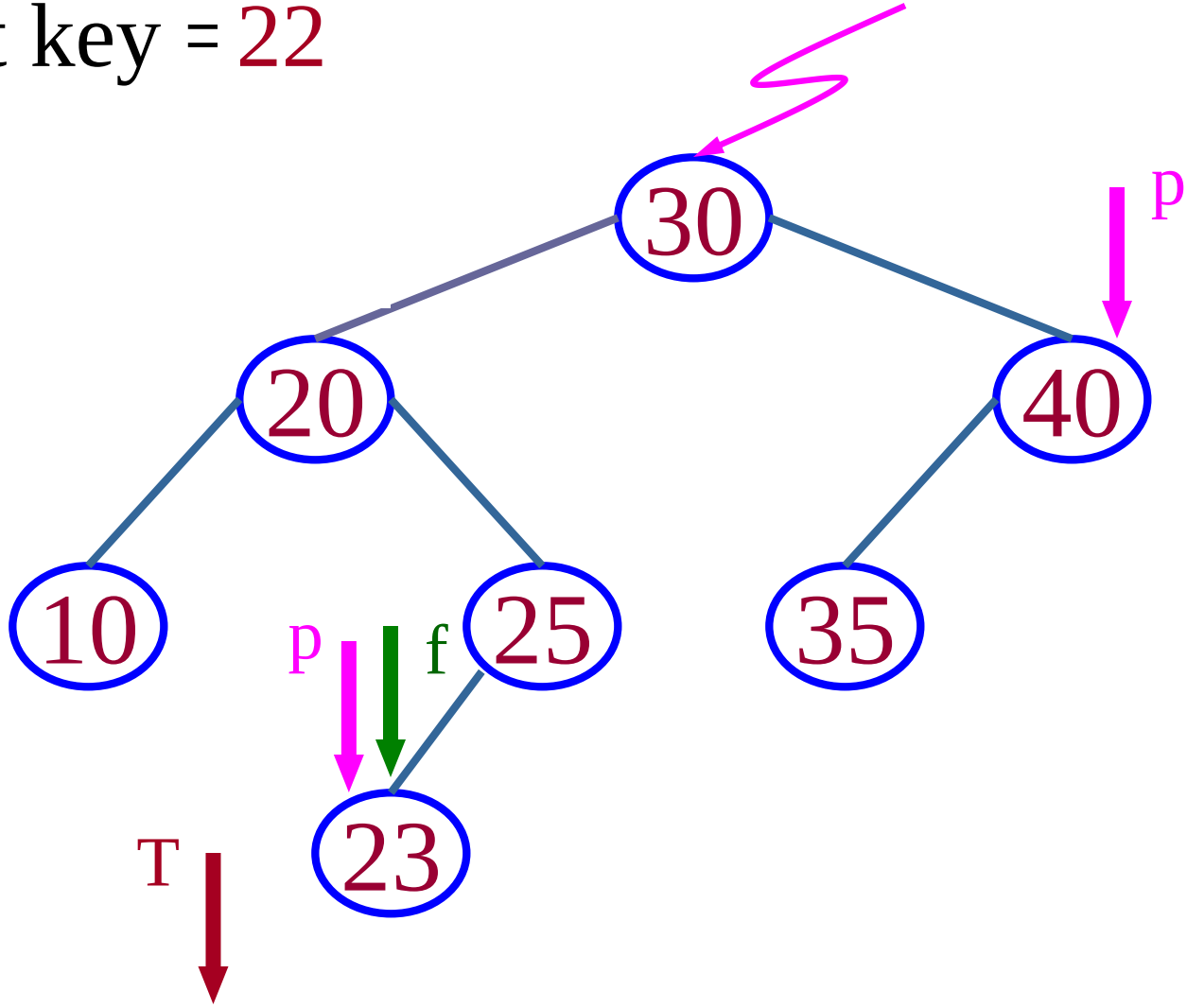
Status SearchBST (BiTree T, KeyType key,
BiTree f, BiTree & p) {

Recursively search for its in the binary search tree pointed to by the root pointer T
// Data element with a keyword equal to key, if found,
// returns the node of the data element pointed to by pointer p, and returns
// The function value is TRUE;
Otherwise, the search will fail and an error message will be returned.
// The last node visited by pointer p along the search path,
// and return the function value FALSE, with pointer f pointing to the current access
// The parent node of the node, with its initial call value set to NULL
… … … …

} // SearchBST

```
if (!T)
    { p = f;  return FALSE; }  // Failed to find
else  if ( EQ(key, T->data.key) )
    { p = T;  return TRUE; }  // Found
else  if ( LT(key, T->data.key) )
    SearchBST (T->lchild, key, T, p );
    // Continue searching in the left subtree
else  SearchBST (T->rchild, key, T, p );
    // Continue searching in the right subtree
```

Set key = 22



3. Binary sort tree insertion algorithm

According to the definition of a dynamic search table, the "insertion" operation is only performed when the search is **unsuccessful**.

If the binary search tree is **empty**, the newly inserted node becomes the **new root node**.
Otherwise, the **newly inserted node** must be a new leaf node, and its insertion position is determined by the search process.

```
Status Insert BST(BiTree &T, ElemType e )
{// When inserting an element, add a node with value e and return
// Return TRUE; otherwise, do not insert and return FALSE
  if (!SearchBST ( T, e.key, NULL, p ))
    {     …     …          }
  else return FALSE;
} // Insert BST
```

```
s = (BiTree) malloc (sizeof (BiTNode));
 // Allocate space for new nodes
s->data = e;
s->lchild = s->rchild = NULL;
if  ( !p )  T = s;     // Insert s as the new root node
else   if ( LT(e.key, p->data.key) )
      p->lchild = s;     //  Insert *s as the left child of *p
else  p->rchild = s;  // Insert *s as the right child of *p
return TRUE;              //  Insertion succeeded
```

4. Delete algorithm of binary sort tree

In contrast to insertion, deletion is performed after a successful search, and requires that the binary search tree characteristics remain after deleting a node from the binary search tree.

There are three cases to consider:

(1) The deleted node is a leaf;

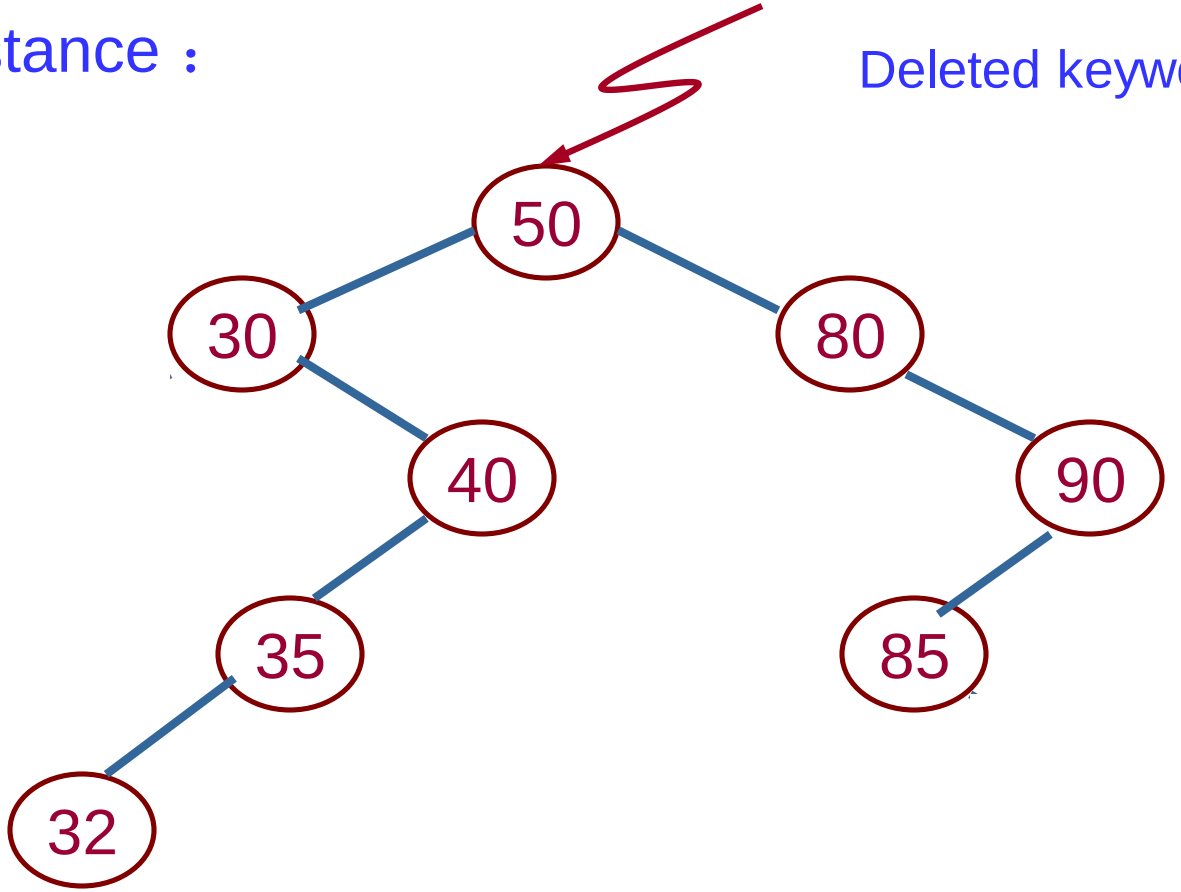
(2) The deleted node has either only a left subtree or only a right subtree;

(3) The deleted node has both a left subtree and a right subtree.

(1) The deleted node is a leaf node

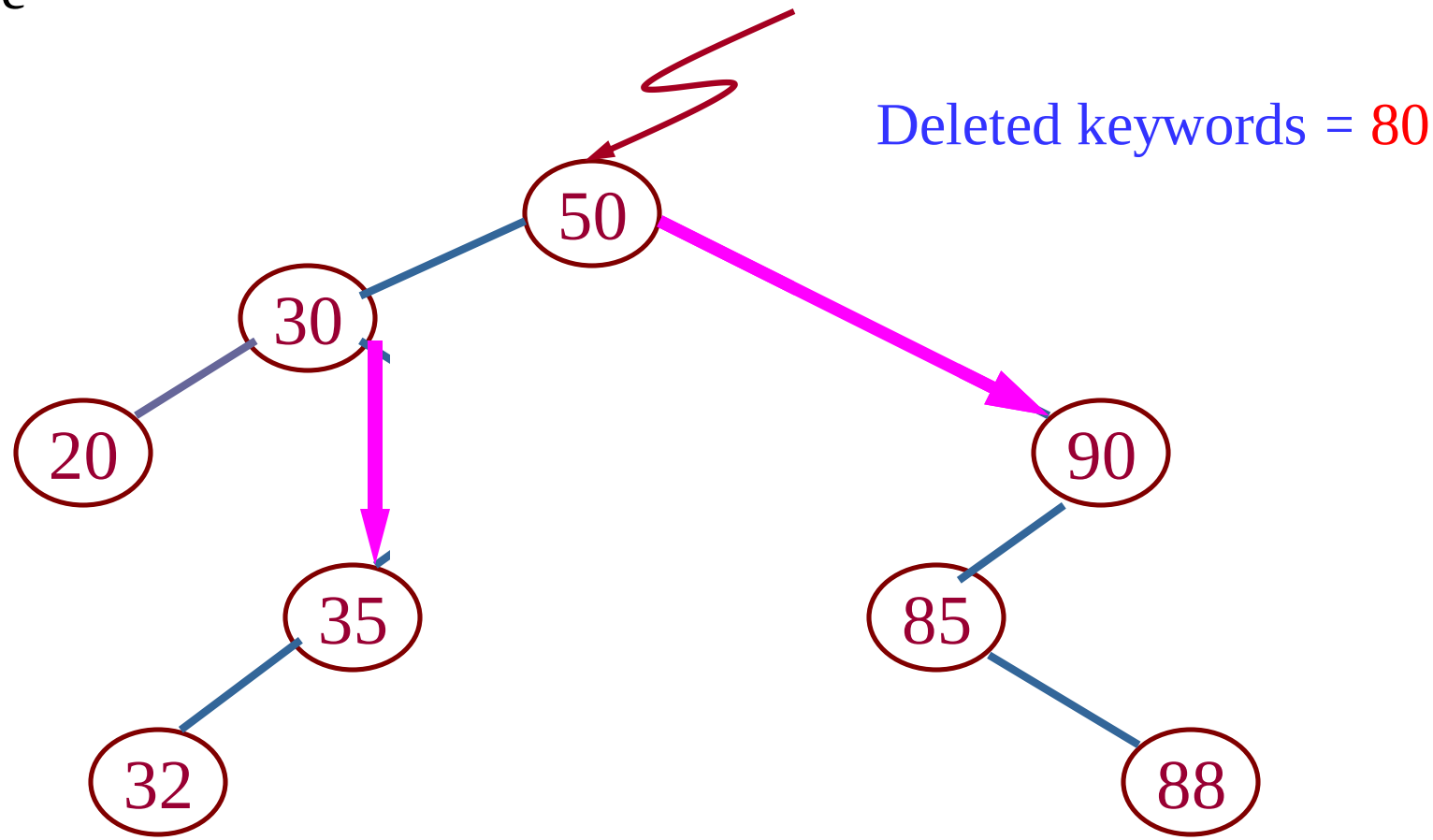
for instance：

Deleted keywords = 88



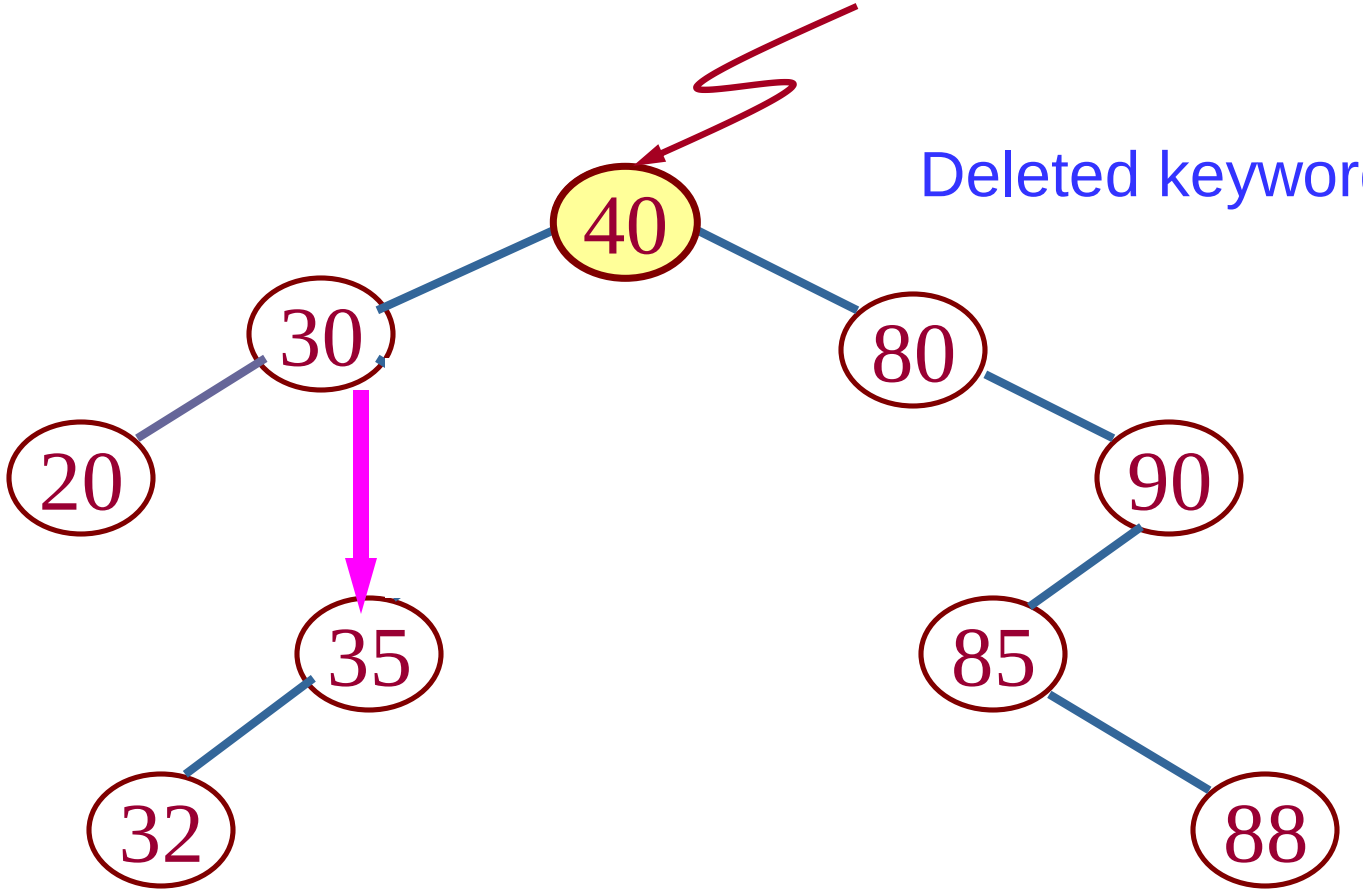
The value of the corresponding pointer field in the parent node is changed to "null"

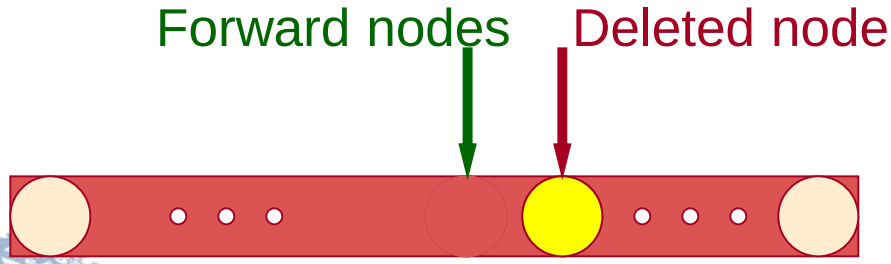
(2) The deleted node has either only a left subtree or only a right subtree



The value of the corresponding pointer field in the parent node is changed to "point to the left or right subtree of the deleted node".

(3) The deleted node has both a left subtree and a right subtree





Replace it with its immediate predecessor and then delete the predecessor node

The algorithm is described as follows:

```
Status DeleteBST (BiTree &T,  KeyType key ) {
// If the binary search tree T contains a node whose key equals key
// Delete the data element node and return
// Returns TRUE if the function value is true, otherwise returns FALSE
  if (!T)  return FALSE;
//No data element exists with the key
  else {     … …        }
} // DeleteBST
```

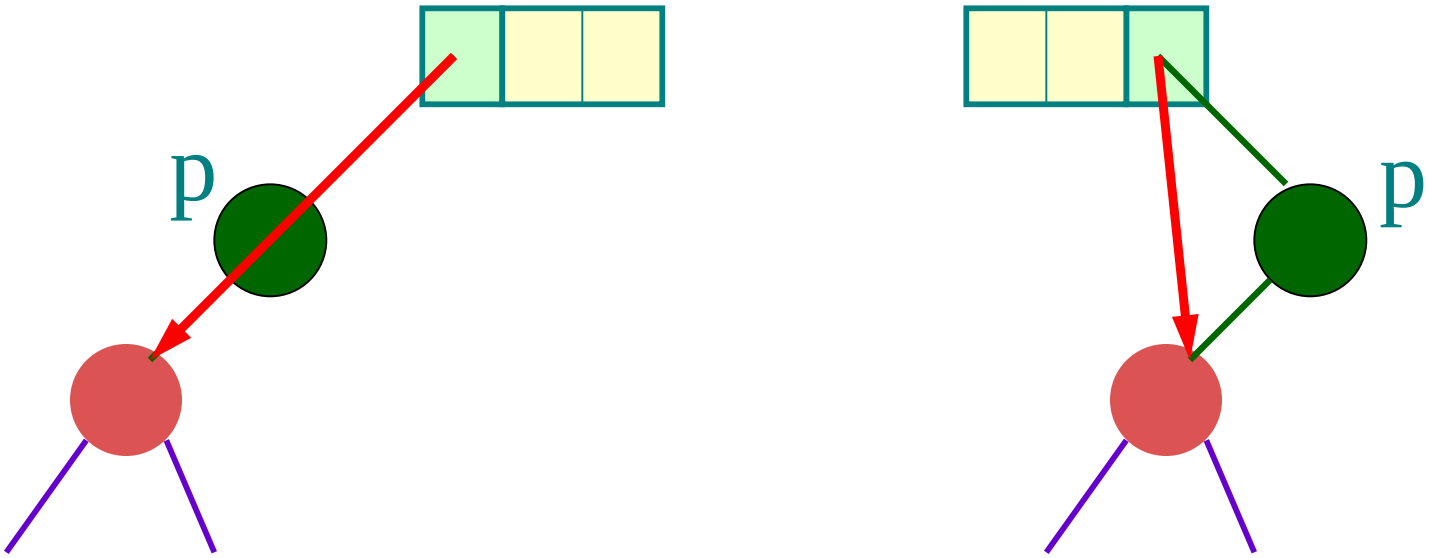
```
if ( EQ (key, T->data.key) )
    {  Delete (T);   return TRUE;  }
// Find data elements with the keyword equal to key
else if ( LT (key, T->data.key) )

    DeleteBST ( T->lchild, key );
  // Continue searching in the left subtree
else  DeleteBST ( T->rchild, key );
      // Continue searching in the right subtree
```

The deletion process is described as follows:

```
void Delete ( BiTree &p ){
  // Remove node p from the binary search tree and reconnect it to its left or right subtree
   if (!p->rchild)  {q = p;  p = p->lchild;
                                free(q);} // p has only left child
   else if (!p->lchild) {q = p;  p = p->rchild;
                                free(q);}// p has only right child
   else {  . . .  . . .  }// p has both left and right children
  return TRUE;
} // Delete
```

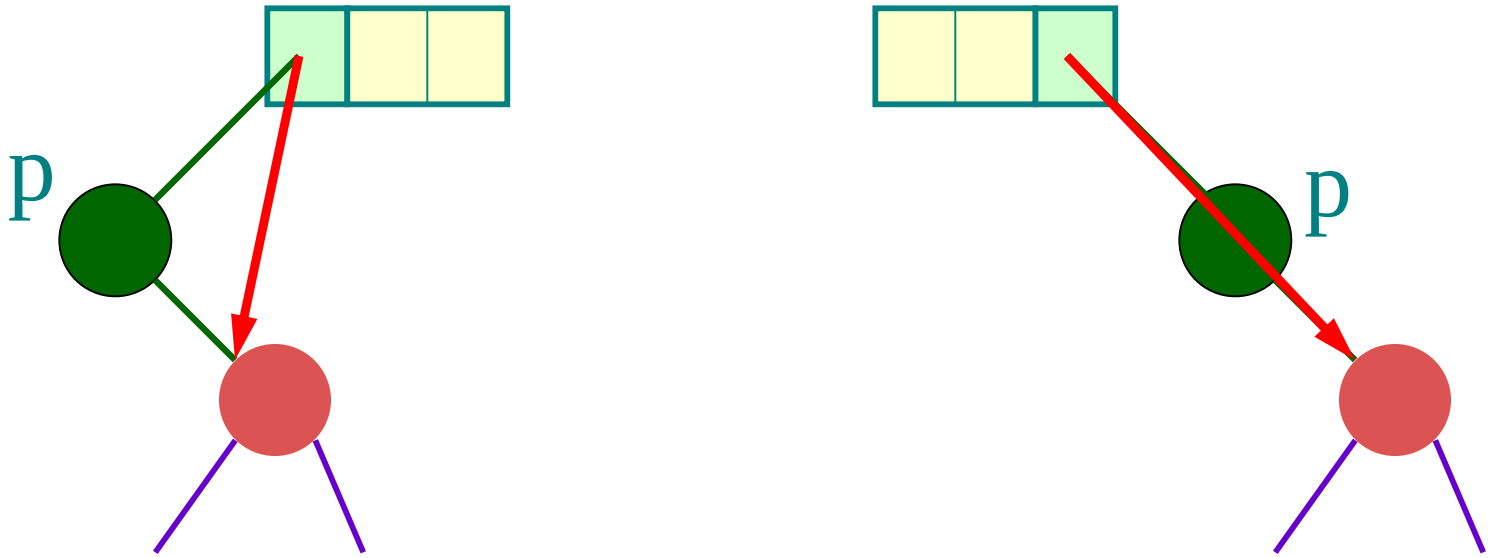
// If the right subtree is empty, simply reconnect its left subtree

```
q = p;  p = p->lchild;  free(q);
```



// The left subtree is empty. Simply reconnect its right subtree.

```
q = p;  p = p->rchild;  free(q);
```



```
// Both left and right subtrees are non-empty

q = p;  s = p->lchild;
while (s->rchild) { q = s;  s = s->rchild; }
// Move left, then right to the end, pointing to the direct predecessor S of the deleted node P

p->data = s->data; //Directly replace the predecessor node p
if (q != p )  q->rchild = s->lchild;//Reattach the right subtree of *q
else  q->lchild = s->lchild;
// q=p indicates that Ps left child S has no right child, then ps left child S
//is the direct predecessor of p. Reconnect the left subtree of q.
free(s);
```

Conclusion：

(1) An **unsorted sequence** (10,18,3, 8, 12,2, 7, 3) can be **transformed into an ordered sequence** by constructing a binary sort tree. The process of building this tree is essentially the sorting process itself.

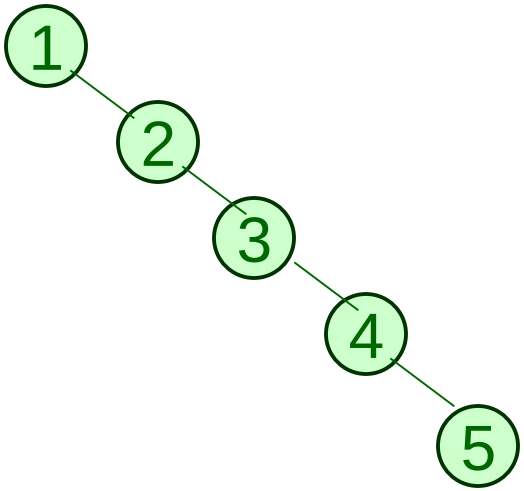
(2) Each newly **inserted node** becomes a leaf node in the binary search tree. During insertion, **no other nodes need to be moved**—only the pointer of the target node needs to be updated from null to non-null. This is analogous to inserting an element into an already sorted sequence without shifting other elements. This characteristic demonstrates that binary search tree structures are more suitable for ordered lists requiring frequent insertions and deletions.

5. Performance analysis

For any given binary search tree, its average search length (ASL) can be calculated using the standard definition. Notably, when constructing different binary search trees from n keywords with identical values, the ASLs may vary significantly, sometimes by a considerable margin.

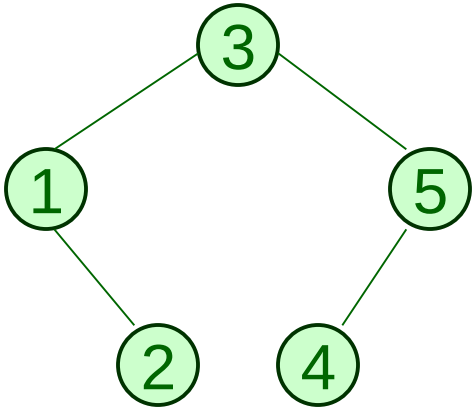
for instance：

by keyword sequence 1, 2, 3, 4, 5
A binary sort tree constructed



ASL =（1+2+3+4+5）/ 5
= 3

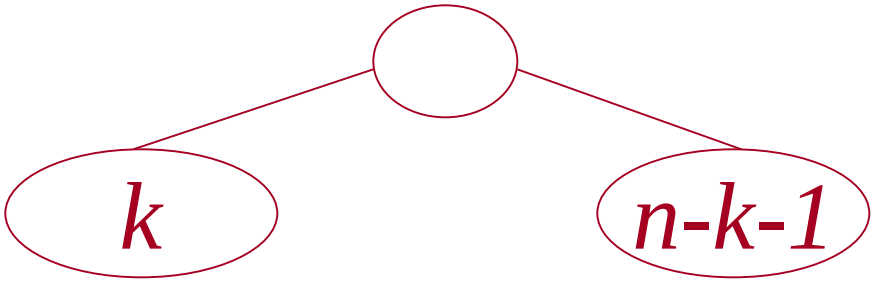
by keyword sequence 3, 1, 2, 5, 4
A binary sort tree constructed



ASL =（1+2+3+2+3）/ 5
= 2.2

Next, let's discuss the average case:

Without generalization, in a sequence of length n, if k keywords are smaller than the first, then n-k-1 keywords must be larger. The binary sort tree constructed from these keywords



The average search length is a function of n and k

$$P(n, k) \qquad (0 \le k \le n-1)。$$

Assuming all n keywords have equal probability of appearing at the first position, the average search length of a binary search tree containing n keywords is:

$$ASL = P(n) = \frac{1}{n}\sum_{k=0}^{n-1} P(n,k)$$

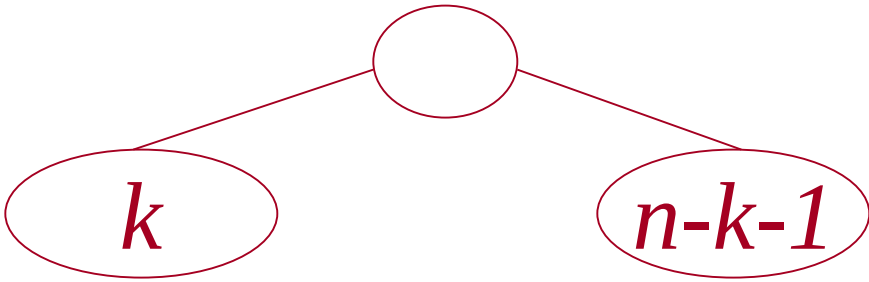
In the case of equal probability search,

$$P(n,k) = \sum_{i=1}^{n} p_i C_i = \frac{1}{n}\sum_{i=1}^{n} C_i$$

$$P(n,k)=\frac{1}{n}\sum_{i=1}^{n}C_i=\frac{1}{n}\left(C_{root}+\sum_{L}C_i+\sum_{R}C_i\right)$$

$$=\frac{1}{n}\left(1+k(P(k)+1)+(n-k-1)(P(n-k-1)+1)\right)$$

$$=1+\frac{1}{n}\left(k\times P(k)+(n-k-1)\times P(n-k-1)\right)$$



from this

$$P(n) = \frac{1}{n}\sum_{k=0}^{n-1}\left(1 + \frac{1}{n}\left(k \times P(k) + (n-k-1) \times P(n-k-1)\right)\right)$$

$$= 1 + \frac{2}{n^2}\sum_{k=1}^{n-1}\left(k \times P(k)\right)$$

Similar to solving a difference equation, this recursive equation has a solution:

$$P(n) = 2\frac{n+1}{n}\log n + C$$

Thank You !